

Categorización de imágenes de dígitos mediante IBL

Minería de datos

Alvaro Bello Garrido

20/12/2025

Índice

Analizar el conjunto de datos de dígitos (Digits Data Set)	3
Visualización de una Imagen Individual	3
Visualización de la Media de Cada Dígito	3
Visualización de la Desviación Estándar de Cada Dígito	3
División de train y test, y normalización de los datos	4
Proyección de los componentes principales	5
Reducción de dimensionalidad	5
Muestreo	6
Predicción con k-neighbors	8
Análisis de resultados (valores fijos)	10
Comparación de algoritmos y búsqueda de valores óptimos	12
Análisis de resultados (valores óptimos)	14
Mejor Pipeline General	14
Análisis de Tendencias de los gráficos	15
Conclusiones	17

Analizar el conjunto de datos de dígitos (Digits Data Set)

Inicialmente, en los primeros bloques de código comenzamos importando las librerías necesarias y cargando el data set de dígitos de la librería sklearn sobre las matrices X e Y. Donde X contiene en cada fila el contenido de la muestra de un dígito representada por matrices de 8x8 (64 elementos), e Y contiene el número que representa cada muestra de X.

Visualización de una Imagen Individual

En este bloque de código se obtiene la muestra 3 del dataset y se remodela en una matriz 8x8 para mostrar mediante el color map “plasma” la muestra como imagen.

Visualización de la Media de Cada Dígito

En este bloque se crea una cuadrícula de subgrafos para mostrar la imagen promedio de cada dígito (del 0 al 9) calculando el valor medio para cada dígito.

Visualización de la Desviación Estándar de Cada Dígito

Este bloque es muy similar al anterior, pero en lugar de calcular la media, calcula y muestra la desviación estándar para cada dígito.

División de train y test, y normalización de los datos

Inicialmente utilizamos la función “train_test_split” para dividir las muestras en los grupos train y test como nos indica el enunciado, y mostramos la cantidad de muestras de cada grupo.

Después realizamos la estandarización de los datos tanto de forma manual como utilizando la librería sklearn:

- Para realizarla de forma manual: obtenemos la media y la desviación estándar de los pixels de todo el conjunto de train; para seguidamente, estandarizar los datos restando a cada pixel de cada muestra su valor medio calculado en “u_train”, y dividiéndolo entre su desviación calculada en “sd_train”.
- Para realizarla utilizando la librería sklearn: creamos una instancia del StandardScaler para que aplique la estandarización de forma más automática empleando la función “fit_transform”.

Proyección de los componentes principales

Reducción de dimensionalidad

Para proyectar las instancias primero hemos de reducir su dimensionalidad (la cual en el dataset es de 64) a una dimensionalidad representable (como 2D o 3D). Para ello utilizaremos los algoritmos sugeridos en el enunciado (PCA y SVD) mediante la función “fit_transform” de las instancias “sklearn.decomposition.PCA” y “sklearn.decomposition.TruncatedSVD”, las cuales inicializamos indicando el número de componentes en 3 (también podríamos reducir la dimensionalidad a 2 componentes, pero decidimos utilizar 3 ya que resulta más preciso).

Además, añadimos los siguientes algoritmos extra:

- Kernel PCA el cual también está incluido en la librería sklearn mediante la instancia “sklearn.decomposition.KernelPCA”. Este algoritmo en vez de realizar separaciones lineales, realiza separaciones más circulares o curvas, lo que puede resultar muy interesante en este proyecto debido a que los números muchas veces se presentan con formas curvas (un claro ejemplo es el 0).
- Y LDA el cual hemos estudiado en clase y también viene incluido en la librería sklearn mediante la instancia “sklearn.discriminant_analysis.LinearDiscriminantAnalysis”, inicializándola a N -1 donde N son los grupos posibles de números (10), resultando en 9.

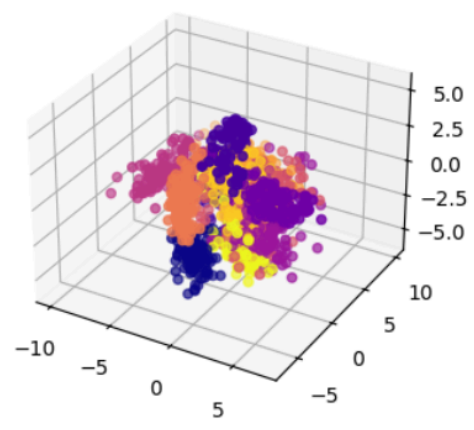
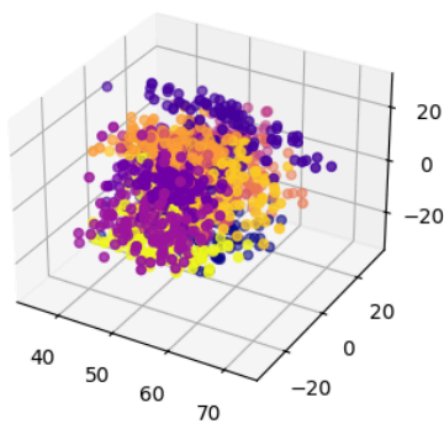
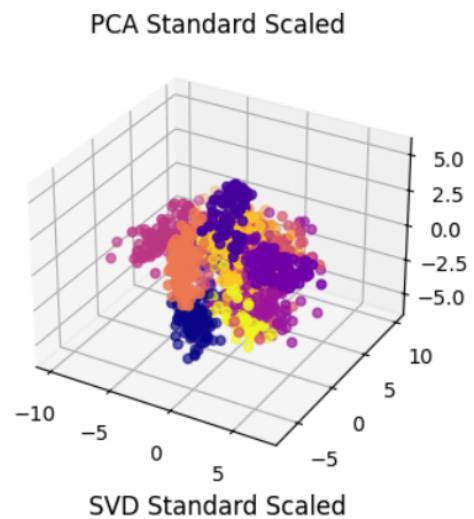
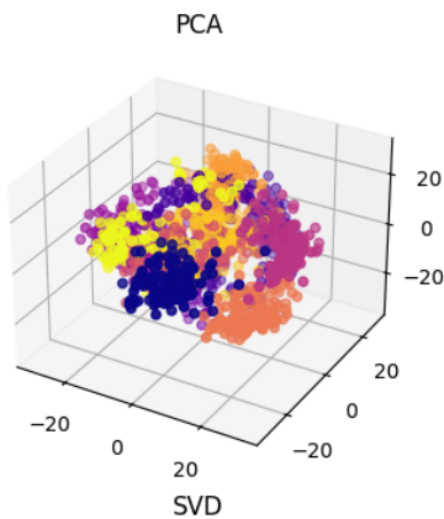
Aplicamos estos algoritmos tanto para los datos sin normalizar como para los normalizados, con el objetivo de comparar resultados.

Muestreo

Para mostrar los datos obtenidos en el apartado anterior creamos dos cuadrículas de subgráficos (axes) con 2 filas y 2 columnas, especificando su tamaño y que serán 3D.

Seguidamente, establecemos un título para cada subgráfico y dibujamos las muestras obtenidas mediante la función scatter utilizando los resultados obtenidos en el apartado anterior.

Podemos observar como en el algoritmo SVD hay una diferencia mucho más notoria que en PCA entre su aplicación sobre los datos normalizados y sin normalizar.



Investigando en la documentación nos damos cuenta que esto se debe a que en el algoritmo PCA se centran los datos automáticamente antes de aplicar el algoritmo, mientras que en SVD no.

TruncatedSVD

```
class sklearn.decomposition.TruncatedSVD(n_components=2, *, algorithm='randomized',  
n_iter=5, n_oversamples=10, power_iteration_normalizer='auto', random_state=None,  
tol=0.0) \[source\]
```

Dimensionality reduction using truncated SVD (aka LSA).

This transformer performs linear dimensionality reduction by means of truncated singular value decomposition (SVD). Contrary to PCA, this estimator does not center the data before computing the singular value decomposition. This means it can work with sparse matrices efficiently.

Predicción con k-neighbors

Para utilizar el clasificador k-neighbors de sklearn realizamos los siguientes pasos para cada grupo de datos obtenido por los anteriores algoritmos:

1. Creamos la instancia "sklearn.neighbors.KNeighborsClassifier" inicializándola con el número de vecinos deseado a considerar por el algoritmo.
2. Entrenamos el algoritmo mediante la función 'fit' con el conjunto de datos de entrenamiento deseado.
3. Realizamos la predicción mediante la función predict (utilizando el conjunto de test normalizado para los algoritmos que se hayan realizado sobre las muestras de entrenamiento normalizadas).
4. Mostramos un reporte de la clasificación para analizar resultados.

Analizando los resultados podemos ver que el algoritmo de reducción de dimensionalidad que nos da mayor precisión para "neighbors=5" es LDA tanto con los datos normalizados como sin.

LDA

	precision	recall	f1-score	support
0	1.00	1.00	1.00	53
1	0.94	1.00	0.97	50
2	0.98	0.98	0.98	47
3	0.94	0.94	0.94	54
4	0.97	0.98	0.98	60
5	0.95	0.95	0.95	66
6	0.98	0.98	0.98	53
7	0.96	0.98	0.97	55
8	0.97	0.88	0.93	43
9	0.97	0.95	0.96	59
accuracy			0.97	540
macro avg	0.97	0.97	0.97	540
weighted avg	0.97	0.97	0.97	540

LDA_SS

	precision	recall	f1-score	support
0	1.00	1.00	1.00	53
1	0.94	1.00	0.97	50
2	0.98	0.98	0.98	47
3	0.94	0.94	0.94	54
4	0.97	0.98	0.98	60
5	0.95	0.95	0.95	66
6	0.98	0.98	0.98	53
7	0.96	0.98	0.97	55
8	0.97	0.88	0.93	43
9	0.97	0.95	0.96	59
accuracy			0.97	540
macro avg	0.97	0.97	0.97	540
weighted avg	0.97	0.97	0.97	540

Análisis de resultados (valores fijos)

Para analizar el rendimiento del clasificador de forma más robusta creamos la función “compute_test” la cual utiliza la técnica de validación cruzada K-Fold para obtener un mejor análisis.

La función divide los datos en cv (por defecto 10) subconjuntos; en cada iteración, entrena el clasificador con la mayoría de los subconjuntos y lo prueba con el restante. La precisión obtenida en cada una de estas pruebas se guarda en una lista, la cual es finalmente devuelta por la función. De esta forma, se obtiene un conjunto de puntuaciones que reflejan la consistencia del rendimiento del modelo, en lugar de una única puntuación que podría ser sensible a una división particular de los datos.

Investigando la documentación de sklearn encontramos que mediante objetos “pipeline” podemos definir una secuencia de pasos de preprocesado de datos y un predictor como elemento final.

Pipeline

```
class sklearn.pipeline.Pipeline(steps, *, transform_input=None, memory=None, verbose=False)
```

[\[source\]](#)

A sequence of data transformers with an optional final predictor.

`Pipeline` allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.

Intermediate steps of the pipeline must be transformers, that is, they must implement `fit` and `transform` methods. The final estimator only needs to implement `fit`. The transformers in the pipeline can be cached using `memory` argument.

Finalmente para analizar los resultados utilizando esta función, utilizaremos el pipeline creado en clase el cual:

1. Normaliza los datos
2. Utiliza LDA para reducir la dimensionalidad
3. Predice empleando el clasificador de K-Neighbors

El resultado devuelto es muy parecido al anterior obtenido con LDA.

```
Final accuracy is 97.38%
```

Comparación de algoritmos y búsqueda de valores óptimos

Para comparar los algoritmos y buscar los valores de configuración óptimos para estos comenzamos definiendo pipelines de todas las configuraciones de algoritmos (PCA, SVD, KPCA y LDA, con y sin normalización de los datos); y rangos de número de vecinos a considerar en la predicción junto a los rangos de número de componentes en dimensionalidad, para buscar los valores de configuración óptimos.

Para realizar la exploración de valores óptimos, investigando en la documentación de sklearn encontramos el algoritmo “GridSearchCV” el cual implementa esta búsqueda.

GridSearchCV

```
class sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None,
n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs',
error_score=nan, return_train_score=False)
```

[\[source\]](#)

Exhaustive search over specified parameter values for an estimator.

Important members are fit, predict.

GridSearchCV implements a “fit” and a “score” method. It also implements “score_samples”, “predict”, “predict_proba”, “decision_function”, “transform” and “inverse_transform” if they are implemented in the estimator used.

Para utilizarlo lo instanciamos inicializándolo con el pipeline seleccionado junto a sus rangos de valores a explorar que definimos en el apartado anterior y mediante la función fit realiza la búsqueda.

Una vez completada la búsqueda guardamos la mejor puntuación, los parámetros utilizados con la mejor puntuación, y un cv con los resultados para después mostrar la evolución de la búsqueda gráficamente. Y repetimos el proceso con todos los pipelines.

Finalmente, para mostrar gráficamente la evolución de los resultados, creamos otra matriz de gráficos y mostramos los resultados de cada pipeline mediante la siguiente secuencia:

1. Recuperación del `param_grid` original: Buscamos el rango ya definido en el apartado anterior del pipeline actual.
2. Identificación de la Clave de `n_components`: Buscamos el nombre de la clave de '`n_components`' del algoritmo actual contenida dentro del rango obtenido en el punto anterior (esta clave será utilizada posteriormente para buscar los resultados del algoritmo en base al número de componentes).
3. Recorrer los resultados por '`n_neighbors`': Para obtener los resultados dentro de '`cv_results_df`' comenzamos obteniendo el contenido dentro de la clave "`param_kneighborsclassifier__n_neighbors`" (la cual obtuvimos analizando la variable en modo debug) y recorreremos los resultados obtenidos en base a los neighbors seleccionados en orden ascendente.
4. Graficar los resultados del neighbor utilizado en base a '`n_components`': Graficamos como un punto la media de los resultados obtenidos por cada '`n_components`' (recuperado el nombre de parámetro del punto 2) para cada '`n_neighbors`'.
5. Configuración de Títulos y Etiquetas de los Ejes: Establecemos el título correspondiente a la gráfica actual y las etiquetas de los ejes.

Análisis de resultados (valores óptimos)

Mejor Pipeline General

Los pipelines PCA, SVD y KPCA, todos sin escalado previo, lograron la puntuación más alta de 0.9865. Estos tres pipelines obtuvieron su mejor rendimiento con `n_neighbors=1` y `n_components=30`. El resto de algoritmos tuvieron puntuaciones muy cercanas pero un poco inferiores.

```
--- All GridSearchCV Results ---

PCA:
  Best Score: 0.9889
  Best Params: {'kneighborsclassifier__n_neighbors': 1, 'pca__n_components': 35}

PCA_SS:
  Best Score: 0.9769
  Best Params: {'kneighborsclassifier__n_neighbors': 4, 'pca__n_components': 25}

SVD:
  Best Score: 0.9889
  Best Params: {'kneighborsclassifier__n_neighbors': 1, 'truncatedsvd__n_components': 35}

SVD_SS:
  Best Score: 0.9769
  Best Params: {'kneighborsclassifier__n_neighbors': 4, 'truncatedsvd__n_components': 25}

KPCA:
  Best Score: 0.9889
  Best Params: {'kernelpca__n_components': 35, 'kneighborsclassifier__n_neighbors': 1}

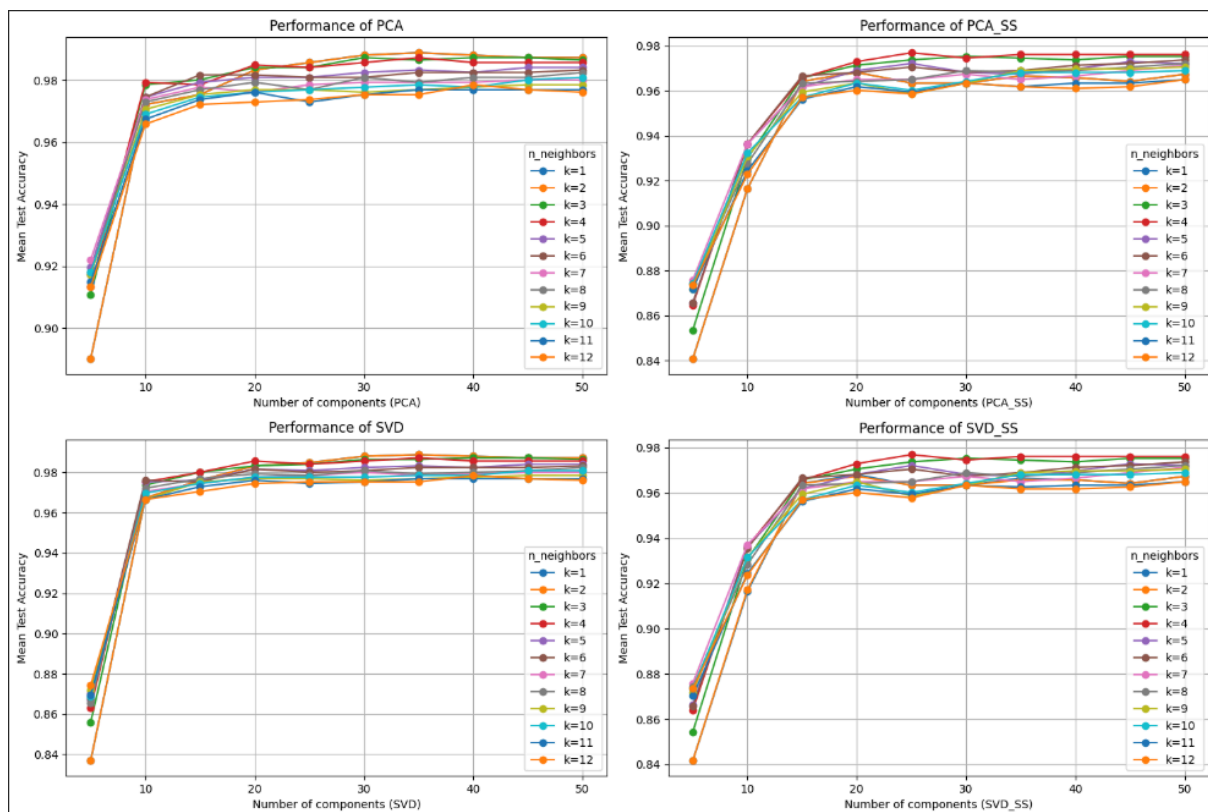
KPCA_SS:
  Best Score: 0.9769
  Best Params: {'kernelpca__n_components': 25, 'kneighborsclassifier__n_neighbors': 4}

LDA:
  Best Score: 0.9689
  Best Params: {'kneighborsclassifier__n_neighbors': 3, 'lineardiscriminantanalysis__n_components': 9}

LDA_SS:
  Best Score: 0.9689
  Best Params: {'kneighborsclassifier__n_neighbors': 3, 'lineardiscriminantanalysis__n_components': 9}
```

Análisis de Tendencias de los gráficos

Revisando los gráficos de rendimiento observamos que a excepción de LDA, todos tienen una evolución muy parecida, bajo la extraña situación de que los modelos con los datos normalizados obtuvieron una puntuación un poco menor que los modelos con los datos sin normalizar, pero lo considero una casualidad causada por la cercanía de la precisión de los resultados, ya acostumbra a ser al revés.



Conclusiones

El efecto de la dimensionalidad es un factor crítico a considerar al trabajar con algoritmos de Machine Learning, especialmente con K-Nearest Neighbors (KNN). En espacios de baja dimensionalidad, como los que hemos reducido aquí con PCA, SVD, KernelPCA y LDA, KNN puede funcionar muy bien porque las distancias entre los puntos son significativas y se pueden encontrar vecinos cercanos de forma confiable. Sin embargo, a medida que la dimensionalidad de los datos aumenta (lo que se conoce como la "maldición de la dimensionalidad"), la distancia entre los puntos se vuelve menos discriminadora. En espacios de alta dimensión, todos los puntos tienden a estar aproximadamente a la misma distancia unos de otros, lo que hace que sea muy difícil para KNN encontrar vecinos verdaderamente "ceranos". Esto puede llevar a que el algoritmo se vuelva ineficiente en términos computacionales y a que su rendimiento de clasificación disminuya drásticamente, ya que la distinción entre un vecino cercano y uno lejano se difumina, haciendo que el modelo se base en información ruidosa o irrelevante.